

Enunciado Evaluacion Parcial 1

Programacion Orientada a Objetos

FORMA B

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1102	Programacion Orientada a Objetos	EA1: Fundamentos de Programacion Orientada a Objetos
Tiempo	Modalidad de Trabajo	Indicadores de logro
3 bloques	Individual	IL 1.1 al IL 1.6

Condiciones de la evaluacion

Del entorno y la entrega

- La evaluacion es de caracter **individual**. No esta permitida ninguna forma de colaboracion entre estudiantes.
- No se permite el acceso a internet ni el uso de telefonos, relojes inteligentes ni ningun otro dispositivo externo.
- El programa debe compilar y ejecutarse correctamente. Una solucion que no compile no permite evidenciar ningun indicador de logro.
- El proyecto debe entregarse comprimido en formato ZIP, con tu nombre completo como nombre del archivo, en la actividad habilitada en AVA antes de que finalice la sesion.
- No se aceptan entregas por ningun otro medio ni fuera del horario establecido. La ausencia de entrega en AVA genera automaticamente la nota minima.

De tu solucion

- El codigo debe organizarse en clases separadas, respetando las convenciones de nomenclatura de Java: `PascalCase` para nombres de clase, `camelCase` para metodos y atributos.

- Puedes crear metodos o clases auxiliares siempre que no contradigan los requerimientos del caso.
 - No se admite el uso de herramientas de inteligencia artificial ni de recursos digitales externos.
 - Cualquier indicio de colaboracion no autorizada, uso de IA o plagio implica la anulacion inmediata de la evaluacion con nota minima de 1.0.
-

Instrucciones de entrega con Git

Antes de comenzar a codificar, debes configurar Git y preparar tu repositorio. Sigue estos pasos en orden:

Paso 1 — Configurar tu identidad en Git (si no lo has hecho previamente)

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu.correo@duocuc.cl"
```

Paso 2 — Clonar el repositorio de tu seccion

Seccion 004D:

```
git clone https://github.com/hachimaki-dev/004D_Desarrollo_orientado_a_Objeto_2026
```

Seccion 002D:

```
git clone https://github.com/hachimaki-dev/002D_desarrollo_orientado_a_objeto_2026
```

Paso 3 — Ingresar al directorio del repositorio clonado

```
cd 004D_Desarrollo_orientado_a_Objeto_2026
```

Ajusta el nombre del directorio segun tu seccion.

Paso 4 — Crear una rama nueva para tu entrega

```
git checkout -b evaluacion1/Nombre_Apellido
```

Ejemplo: `git checkout -b evaluacion1/Carlos_Orellana`

Paso 5 — Desarrollar tu solucion dentro de esta rama

Crea tus archivos `.java` y trabaja tu solucion normalmente.

Paso 6 — Guardar tus cambios en un commit

```
git add .  
git commit -m "Entrega EP1 - Nombre Apellido - Confirmo que esta es mi entrega f
```

Paso 7 — Publicar tu rama en el repositorio remoto

```
git push origin evaluacion1/Nombre_Apellido
```

Importante: Además de publicar tu rama en Git, debes comprimir tu proyecto en formato ZIP (con tu nombre completo como nombre del archivo) y subirlo al AVA antes de que finalice la sesión. Ambas entregas son obligatorias.

Caso de negocio

Una biblioteca municipal necesita un sistema de consola en Java para gestionar el préstamo de materiales de lectura. La biblioteca maneja dos tipos de materiales: **libros** y **revistas**. Ambos comparten datos generales como título, autor y cantidad de ejemplares disponibles, pero se diferencian en sus atributos específicos y en la cantidad de días permitidos para el préstamo.

El sistema debe permitir registrar materiales, listar el catálogo, buscar materiales por título, prestar ejemplares y ver un resumen del catálogo. Todas las entradas del usuario deben estar protegidas contra errores de ingreso.

Estructura de clases

El siguiente diagrama muestra la jerarquía de clases a implementar. La clase `Material` es **abstracta** y define la estructura base. Las clases `Libro` y `Revista` heredan de ella y sobrescriben el cálculo de los días de préstamo.

Material (abstracta)

```
- titulo: String  
- autor: String  
- cantidadDisponible: int
```

```
+ Material(titulo, autor, cantidadDisponible)  
+ getTitulo(): String  
+ getAutor(): String  
+ getCantidadDisponible(): int  
+ setCantidadDisponible(int): void  
+ calcularDiasPrestamo(): int  
+ mostrarInfo(): String
```

▲ extends

Libro

```
- numPaginas: int
```

```
+ Libro(titulo, autor, cantidadDisponible, numPaginas)  
+ getNumPaginas(): int  
+ calcularDiasPrestamo(): int  
+ mostrarInfo(): String
```

Revista

```
- mesPublicacion: String
```

```
+ Revista(titulo, autor, cantidadDisponible, mesPublicacion)  
+ getMesPublicacion(): String  
+ calcularDiasPrestamo(): int  
+ mostrarInfo(): String
```

Nota: Los días de préstamo se manejan como `int` (días enteros). La cantidad disponible representa el número de ejemplares físicos que la biblioteca tiene para prestar.

Requerimientos funcionales

A continuacion se detalla cada requerimiento. Las etiquetas **IL** indican que indicador de logro se evalua en cada parte.

R1 — Comentarios de paradigma **IL 1.1**

Al inicio de la clase `Main.java`, incluye un bloque de comentarios que explique:

- Que paradigma de programacion se esta usando y sus características principales.
- Al menos **2 diferencias concretas** entre el paradigma orientado a objetos (Java) y el paradigma que usabas antes (Python funcional). Por ejemplo: tipado, estructura del programa, compilacion vs. interpretacion, o forma de organizar el codigo.

R2 — Clase abstracta Material **IL 1.3** **IL 1.4**

Implementa la clase abstracta `Material` segun el diagrama UML:

- Atributos **privados**: `titulo` (String), `autor` (String), `cantidadDisponible` (int).
- Constructor que reciba los 3 atributos.
- Getters para los tres atributos y setter para `cantidadDisponible`.
- Metodo abstracto `calcularDiasPrestamo()` que retorna `int`.
- Metodo `mostrarInfo()` que retorna un `String` con el titulo, autor, dias de prestamo y cantidad disponible. Este metodo puede ser sobrescrito por las clases hijas para agregar informacion adicional.

R3 — Clase Libro

IL 1.3

IL 1.4

Hereda de `Material`. Representa un libro de la biblioteca.

- Atributo privado adicional: `numPaginas` (int).
- Constructor que reciba los 4 parametros (3 del padre + `numPaginas`). Usa `super()`.
- Getter para `numPaginas`.
- `calcularDiasPrestamo()`: si `numPaginas` es mayor a **300**, retorna **14** dias. En caso contrario, retorna **7** dias.
- `mostrarInfo()`: sobrescribe (`@Override`) el metodo del padre. Agrega el tipo (`"Libro"`) y el numero de paginas al texto retornado.

R4 — Clase Revista

IL 1.3

IL 1.4

Hereda de `Material`. Representa una revista de la biblioteca.

- Atributo privado adicional: `mesPublicacion` (String).
- Constructor que reciba los 4 parametros (3 del padre + `mesPublicacion`). Usa `super()`.
- Getter para `mesPublicacion`.
- `calcularDiasPrestamo()`: retorna siempre **3** dias.
- `mostrarInfo()`: sobrescribe (`@Override`) el metodo del padre. Agrega el tipo (`"Revista"`) y el mes de publicacion al texto retornado.

R5 — Menu de consola con estructuras de control IL 1.2

La clase `Main` debe presentar un menu de consola que se repita hasta que el usuario elija salir. Usa un bucle `while` y `switch` para controlar las opciones:

```
≡ BIBLIOTECA MUNICIPAL ≡  
1. Registrar material  
2. Listar catalogo  
3. Buscar material por titulo  
4. Prestar material  
5. Resumen del catalogo  
6. Salir  
Seleccione una opcion: _
```

Las opciones invalidas deben mostrar un mensaje y volver al menu sin que el programa se detenga.

R6 — Registrar material (con submenu) IL 1.2 IL 1.6

Al seleccionar "Registrar material", el sistema muestra un submenu para elegir entre Libro y Revista. Luego solicita los datos correspondientes y agrega el material a una coleccion `ArrayList<Material>`.

- Para libros, se solicita el numero de paginas como un valor entero positivo.
- Para revistas, se solicita el mes de publicacion como texto.
- La cantidad disponible debe ser un numero entero positivo.

R7 — Listar catalogo (polimorfismo en accion) IL 1.5 IL 1.6

Recorre la coleccion `ArrayList<Material>` e imprime la informacion de cada material usando `mostrarInfo()`.

Cada material se muestra con su formato propio (libro o revista) sin necesidad de hacer `instanceof`. Esto es polimorfismo: el mismo metodo produce resultados distintos segun el tipo real del objeto.

Si la lista esta vacia, mostrar un mensaje indicando que no hay materiales registrados.

R8 — Buscar material por título

IL 1.2

IL 1.6

El usuario ingresa un texto y el sistema busca materiales cuyo título contenga ese texto (busqueda parcial, sin distinguir mayusculas/minusculas). Muestra todos los resultados encontrados o un mensaje si no hay coincidencias.

R9 — Prestar material (descontar disponibilidad)

IL 1.2

IL 1.5

IL 1.6

El sistema lista los materiales con su índice. El usuario selecciona un material para prestar. El sistema valida:

- Que el número de material sea válido (exista en la lista).
- Que la cantidad disponible sea mayor a 0.

Si el préstamo es válido, descuenta 1 unidad de `cantidadDisponible` y muestra un comprobante con el título del material, los días de préstamo asignados (calculados con `calcularDiasPrestamo()`) y los ejemplares restantes.

R10 — Resumen del catálogo

IL 1.2

IL 1.5

IL 1.6

Muestra un resumen que incluya:

- Total de materiales registrados.
- Cantidad de libros y cantidad de revistas (usa `instanceof`).
- Total de ejemplares disponibles: la suma de `cantidadDisponible` de todos los materiales.

R11 — Manejo de excepciones IL 1.5

Toda entrada numerica del usuario debe estar protegida con `try-catch` . El programa **nunca debe detenerse** por un error de ingreso. Situaciones que deben manejarse:

- El usuario escribe texto cuando se espera un numero (cantidad disponible, numero de paginas, indice de material).
- El usuario ingresa un indice fuera de rango al prestar.
- El usuario intenta prestar un material con 0 ejemplares disponibles.
- El usuario ingresa una opcion de menu que no existe.

En cada caso, muestra un mensaje claro al usuario y vuelve al menu sin interrumpir la ejecucion.

Ejemplo de ejecucion esperada

Este ejemplo muestra como deberia verse la interaccion con el programa. Tu implementacion debe producir una salida similar (no necesariamente identica en formato).

≡ BIBLIOTECA MUNICIPAL ≡

1. Registrar material
2. Listar catalogo
3. Buscar material por titulo
4. Prestar material
5. Resumen del catalogo
6. Salir

Seleccione una opcion: 1

--- Tipo de material ---

1. Libro
2. Revista

Seleccione: 1

Titulo: Cien anios de soledad

Autor: Gabriel Garcia Marquez

Cantidad disponible: 5

Numero de paginas: 420

[OK] Libro registrado.

Seleccione una opcion: 1

--- Tipo de material ---

1. Libro
2. Revista

Seleccione: 2

Titulo: National Geographic

Autor: Varios

Cantidad disponible: 10

Mes de publicacion: Septiembre 2026

[OK] Revista registrada.

Seleccione una opcion: 2

--- CATALOGO ---

[1] Libro | Cien anios de soledad | Autor: Gabriel Garcia Marquez | Paginas: 420

[2] Revista | National Geographic | Autor: Varios | Mes: Septiembre 2026 | Presta

Seleccione una opcion: 4

--- PRESTAR MATERIAL ---

[1] Cien anios de soledad (Disponibles: 5)

[2] National Geographic (Disponibles: 10)

Ingrese numero de material: 1

Prestamo realizado.

Material: Cien anios de soledad

Dias de prestamo: 14

Ejemplares restantes: 4

Seleccione una opcion: 5

--- RESUMEN DEL CATALOGO ---

Total de materiales: 2

Libros: 1

```
Revistas: 1
Total de ejemplares disponibles: 14

Seleccione una opcion: 3

--- BUSCAR MATERIAL ---
Ingrese texto a buscar: national
Resultados:
    Revista | National Geographic | Autor: Varios | Mes: Septiembre 2026 | Prestamo

Seleccione una opcion: 4
Ingrese numero de material: abc
Error: Debe ingresar un numero valido.

Seleccione una opcion: 6
Programa finalizado.
```

Mapa de indicadores de logro

Esta tabla resume donde se evalua cada indicador. Usala como checklist antes de entregar.

IL	Indicador	Donde se evidencia
IL 1.1	Identificar el paradigma OO y diferenciarlo de otros	Comentarios en <code>Main.java</code> (R1)
IL 1.2	Estructuras de control y metodos con parametros y retorno	Menu while/switch (R5), submenu (R6), busqueda (R8), prestamo (R9)
IL 1.3	Clases con encapsulamiento y colaboracion	Atributos privados, getters/setters, constructores en <code>Material</code> , <code>Libro</code> , <code>Revista</code> (R2, R3, R4)
IL 1.4	Herencia, clases abstractas, jerarquias de tipos	Clase abstracta <code>Material</code> , herencia con <code>extends</code> , <code>super()</code> , metodo abstracto (R2, R3, R4)
IL 1.5	Polimorfismo y manejo de excepciones	Recorrido polimorfico del ArrayList (R7), <code>@Override</code> de metodos (R3, R4), try-catch (R11), instanceof en resumen (R10)
IL 1.6	Colecciones para almacenar y gestionar datos	<code>ArrayList<Material></code> como catalogo, agregar, iterar, buscar, modificar disponibilidad (R6, R7, R8, R9, R10)

Antes de entregar

Debe funcionar

El programa compila con `javac *.java` y se ejecuta con `java Main` sin errores.

Cada opcion del menu hace lo que se pide.

Los metodos sobrescritos (`@Override`) funcionan correctamente en cada subclase.

No debe romperse

Escribir "abc" cuando se pide un numero.

Intentar prestar un material con 0 ejemplares disponibles.

Buscar un material cuando la lista esta vacia.

Seleccionar una opcion de menu inexistente.

Archivos a entregar

Comprime en formato **ZIP** todos los archivos `.java` de tu proyecto. Como minimo debes tener:

- `Main.java`
- `Material.java`
- `Libro.java`
- `Revista.java`

Importante: No se permite el uso de herramientas de IA, acceso a internet, ni dispositivos externos durante la evaluacion. Cualquier indicio de colaboracion no autorizada o uso de IA implica la anulacion de la evaluacion con nota minima 1.0.

Lee bien cada requerimiento, planifica antes de codificar y prueba tu programa antes de entregar.

